ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ

ΑΝΑΦΟΡΑ 1ΟΥ ΕΡΓΑΣΤΗΡΙΟΥ

Ομάδα:

Χαράλαμπος Κουνναπής ΑΜ:5401

Στέλιος Σάββα ΑΜ:5404

Παναγιώτης Χριστοδούλου ΑΜ:5501

Γενική ιδέα:

Προσθέσαμε την χρήση mutex το οποίο χρησιμοποιείται για τον συγχρονισμό νημάτων (threads) και την αποφυγή συγκρούσεων, οπού πολλαπλά νήματα προσπαθούν να διαβάσουν(**read**) και να γράψουν (**write**) δεδομένα ταυτόχρονα σε μια κοινόχρηστη μνήμη. Δηλαδή ενσωματώσαμε ένα mutex στον κώδικα DB (db.c και db.h) και προσθέσαμε κώδικα για το κλείδωμα και ξεκλείδωμα του mutex . Επιπλέων προσθέσαμε μια νέα εντολή “write_read” στο bench.c οπού χρησιμοποιείτε στο τερματικό η οποία μας δίνει την δυνατότητα να συνδυάσουμε της λειτουργίες **write** και **read** με διάφορα ποσά και ποσοστά.

Κώδικας db.c:

db.c

```
#include <string.h>
#include <assert.h>
#include "db.h"
#include "indexer.h"
#include "utils.h"
#include "log.h"

DB* db_open_ex(const char* basedir, uint64_t cache_size)
{
    DB* self = calloc(1, sizeof(DB));

    if (!self)
        PANIC("NULL allocation");

    strncpy(self->basedir, basedir, MAX_FILENAME);
    self->sst = sst_new(basedir, cache_size);

    Log* log = log_new(self->sst->basedir);
    self->memtable = memtable_new(log);

    pthread_mutex_init(&self->db_mutex, NULL);  //arxikopoihsi mutex

    return self;
}

DB* db_open(const char* basedir)
{
    return db_open_ex(basedir, LRU_CACHE_SIZE);
}

void db_close(DB *self)
{
    INFO("Closing database %d", self->memtable->add_count);

    if (self->memtable->list->count > 0)
    {
        sst_merge(self->sst, self->memtable);
        skiplist_release(self->memtable->list);
        self->memtable->list = NULL;
    }

    sst_free(self->sst);
    log_remove(self->memtable->log, self->memtable->lsn);
    log_free(self->memtable->log);
    memtable_free(self->memtable);

    pthread_mutex_destroy(&self->db_mutex);     //katastrofi mutex
```

```
    free(self);
}

int db_add(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->db_mutex);      // kleidoma tou mutex
    if (memtable_needs_compaction(self->memtable))
    {
        INFO("Starting compaction of the memtable after %d insertions and %d deletions",
             self->memtable->add_count, self->memtable->del_count);
        sst_merge(self->sst, self->memtable);
        memtable_reset(self->memtable);
    }


    pthread_mutex_unlock(&self->db_mutex);      // xeklidoma tou mutex(unlock)

    return memtable_add(self->memtable, key, value);;
}

int db_get(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->db_mutex);        // kleidoma tou mutex

    if (memtable_get(self->memtable->list, key, value) == 1) {
        pthread_mutex_unlock(&self->db_mutex);  // kleidoma tou mutex prin tin epistrofi
                                                //se periptosi pou to key vrethi sto memtable
        return 1;
    }
    pthread_mutex_unlock(&self->db_mutex);      // xeklidoma tou mutex

    return sst_get(self->sst, key, value);
}

int db_remove(DB* self, Variant* key)
{
    pthread_mutex_lock(&self->db_mutex);        // kleidoma tou mutex
    pthread_mutex_unlock(&self->db_mutex);      // xeklidoma tou mutex

    return memtable_remove(self->memtable, key);
}

DBIterator* db_iterator_new(DB* db)
{
    DBIterator* self = calloc(1, sizeof(DBIterator));
    self->iterators = vector_new();
    self->db = db;
```

1. Στο function “**db_open_ex**” προσθέσαμε την αρχικοποίηση του mutex.Στην συνέχεια στο function **“db_close”** έχουμε βάλει την εντολή για την καταστροφή του mutex. Ακόμη στα function’s του **“db_add”** , **“db_get”** και **“db_remove”** έχουμε προσθέσει τις εντολές για το κλείδωμα και ξεκλείδωμα του mutex με επιπρόσθετη εντολή στο **“db_get”** για τον αν βρεθεί το κλειδί στο memtable για να κλειδώσει το mutex πριν την επιστροφή. Ουσιαστικά μόνο ένα νήμα (**thread**) μπορεί κάνει ανάγνωση **(read**) ή γραφή(write) από την βάση δεδομένων (**data base**) την ιδιά ώρα για αποφυγή σύγκρουσης μεταξύ τους.

2. Δεν υπάρχει τροποποίηση ή πρόσθεση στον υπόλοιπο κώδικα.

3. Στον κώδικα που έχει προστεθεί, υπάρχουν σχόλια στα δεξιά.

Κώδικας db.h:

db.c

```
#ifndef __DB_H__
#define __DB_H__

#include <pthread.h>    //prosthiki vivliothikis gia mutex
#include "indexer.h"
#include "sst.h"
#include "variant.h"
#include "memtable.h"
#include "merger.h"

typedef struct _db {
//    char basedir[MAX_FILENAME];
    char basedir[MAX_FILENAME+1];
    SST* sst;
    MemTable* memtable;
    pthread_mutex_t db_mutex;   //dilosi metavliti typou pthread_mutex_t
} DB;
```

1. Στον κώδικα **“db.h”** προσθέσαμε την βιβλιοθήκη **“pthread.h”** για να έχουμε πρόσβαση τον τύπο δεδομένων. Στην συνέχεια προσθέσαμε στην function **“struct_db”** την δήλωση ενός αντικειμένου mutex.
2. Ο υπόλοιπος κώδικας δεν έχει καμία άλλη αλλαγή ή πρόσθεση.

<u>Κωδικας bench.h:</u>

bench.h

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdint.h>
#include <string.h>
#include <time.h>
#include <sys/time.h>

#define KSIZE (16)
#define VSIZE (1000)

#define LINE "+-----------------------------+----------------+------------------------------+-------------------+\n"
#define LINE1 "-----------------------------------------------------------------------------------------------------\n"

long long get_ustime_sec(void);
void _random_key(char *key,int length);

void _write_test(long int count, int random);   //dilosi sinartisis me 2 parametrous
void _read_test(long int count, int random);    //gia na mborei na kalestei stin bench.c
                                                //tis sinartisis pou vriskonte stin kiwi.c
```

Στο **“bench.h”** δηλώσαμε δυο συναρτήσεις από την **“kiwi.c”** για να υπάρξει πρόσβαση το αρχείο **“bench.c”** (δοκιμή write και read). Η πρώτη παράμετρος (long int count) στο **_write_test** καθορίζει πόσες τιμές θα εισαχθούν στην βάση δεδομένων, ενώ για την **_read_test** καθορίζει πόσες αναγνώσεις θα γίνουν.

Η δεύτερη παράμετρος (int random) χρησιμοποιείται σαν flag που όταν βρίσκετε στην τιμή μηδέν(0) η λειτουργίες εκτελούνται ακολουθιακά (sequential) ενώ όταν βρίσκετε στην τιμή ένα(1) εκτελείτε με τυχαία σειρά (random).

Κώδικας bench.c:

bench.c

```
#include "bench.h"

void _random_key(char *key,int length) {
        int i;
        char salt[36]= "abcdefghijklmnopqrstuvwxyz0123456789";

        for (i = 0; i < length; i++)
                key[i] = salt[rand() % 36];
}

void _print_header(int count)
{
        double index_size = (double)((double)(KSIZE + 8 + 1) * count) / 1048576.0;
        double data_size = (double)((double)(VSIZE + 4) * count) / 1048576.0;

        printf("Keys:\t\t%d bytes each\n",
                        KSIZE);
        printf("Values: \t%d bytes each\n",
                        VSIZE);
        printf("Entries:\t%d\n",
                        count);
        printf("IndexSize:\t%.1f MB (estimated)\n",
                        index_size);
        printf("DataSize:\t%.1f MB (estimated)\n",
                        data_size);

        printf(LINE1);
}

void _print_environment()
{
        time_t now = time(NULL);

        printf("Date:\t\t%s",
                        (char*)ctime(&now));

        int num_cpus = 0;
        char cpu_type[256] = {0};
        char cache_size[256] = {0};

        FILE* cpuinfo = fopen("/proc/cpuinfo", "r");
        if (cpuinfo) {
                char line[1024] = {0};
                while (fgets(line, sizeof(line), cpuinfo) != NULL) {
                        const char* sep = strchr(line, ':');
                        if (sep == NULL || strlen(sep) < 10)
                                continue;
```

bench.c

```
                        char key[1024] = {0};
                        char val[1024] = {0};
                        strncpy(key, line, sep-1-line);
                        strncpy(val, sep+1, strlen(sep)-1);
                        if (strcmp("model name", key) == 0) {
                                num_cpus++;
                                strcpy(cpu_type, val);
                        }
                        else if (strcmp("cache size", key) == 0)
                                strncpy(cache_size, val + 1, strlen(val) - 1);
                }

                fclose(cpuinfo);
                printf("CPU:\t\t%d * %s",
                                num_cpus,
                                cpu_type);

                printf("CPUCache:\t%s\n",
                                cache_size);
        }
}

int main(int argc,char** argv)
{
        long int count;

        srand(time(NULL));

        if (argc >= 5 && strcmp(argv[1], "write_read") == 0){   //elenxos an i eisodos sto termatiko einai write_read
        int r = 0;                                              //r einai flag gia random input r=0 false r=1 true
        struct timeval start_time, end_time, write_start, write_end, read_start, read_end;      //dilosi metavliton gia xrono

        double total_time, write_time, read_time;       //dilosi metavliton gia ypologismo total_time programmatos

        gettimeofday(&start_time, NULL);        //katagrafi arxikis stigmis enarxis proggramatos kai apothikeusi stin metavliti start_time

        count = atoi(argv[2]);          //metatropi array se integer(arithmos leitourgeion pou eisagei o xristis)
        int write_percent = atoi(argv[3]);      //metatropi atoi(pososto pou theli na kanei write)
        int read_percent = atoi(argv[4]);       //metatropi atoi(pososto pou theli na kanei write)
        if (argc >= 6 && strcmp(argv[5], "random") == 0) {      //elenxos an o xristis theli random leitourgia
                r = 1;  //flag isoute me true(1) ara random
        }

        _print_header(count);   //ektiposi epikefalidas apo to function
        _print_environment();   //ektiposi pliroforion apo to function

        long int write_count = (count * write_percent) / 100; //ypologismos arithmou leitourgion me vasi to pososto pou evale o user
        long int read_count = (count * read_percent) / 100;   //ypologismos arithmou leitourgion me vasi to pososto pou evale o user
        printf("Performing %ld write operations (%d%%)...\n", write_count, write_percent);
        gettimeofday(&write_start, NULL);  //katagrafi xronou stin enarxi write
```

bench.c

```c
        pid_t pid = fork();     //dimiourgia child
        if (pid == 0) {         //enarxi child
                _write_test(write_count, r);    //ektelesi tou _write_test (kiwi.c) me to ypologismeno arithmo kai flag gia random
                exit(0);
        } else if (pid > 0) {   //diergasia parent
                int status;     // dilosi metavliti gia apothikeufsi katastasis termatismou child
                waitpid(pid, &status, 0); //perimenoume ton termatismo tou child

                gettimeofday(&write_end, NULL);  //katagrafi chronou stin lixi tou write
                write_time = (write_end.tv_sec - write_start.tv_sec) + (write_end.tv_usec - write_start.tv_usec) / 1000000.0; //ipologismos chronou write se sec

                gettimeofday(&read_start, NULL);  //katagrafi arxikis stigmis enarxis read kai apothikeusi stin metavliti read_start

                pid = fork(); //dimiourgia neas dieragsias gia to read

                if (pid == 0) {                                 //enarxi diergasias child
                        printf("Performing %ld read operations (%d%%)...\n", read_count, read_percent);

                        if (read_count > write_count && !r) {    //elechos an to read eimai megalitero apo to write
                                printf("Limiting read operations to %ld (number of keys written)\n", write_count); //tiposi minimatos periorismou
                                read_count = write_count;
                        }

                        _read_test(read_count, r); //ektelesi tou _read_test (kiwi.c) me ton arithmo kai to flag
                        exit (0);                  //termatismos diergasias child
                } else if (pid > 0) {      //parent process

                waitpid(pid, &status, 0);  //anamoni gia to termatismo diergasias child

                gettimeofday(&read_end, NULL);  //katagrafi chronou stin lixi read
                read_time = (read_end.tv_sec - read_start.tv_sec) + (read_end.tv_usec - read_start.tv_usec) / 1000000.0; //ipologismos chronou read se sec

                gettimeofday(&end_time, NULL);  //katagrafi sinolikou chronou programmatos
                total_time = (end_time.tv_sec - start_time.tv_sec) + (end_time.tv_usec - start_time.tv_usec) / 1000000.0; //ipologismos se sec

                printf(LINE1);                      //pernoume tin etoimi dilosi grammis stin bench.h
                printf("|%s-Write\t(done:%ld): %.6f sec/op; %.1f writes/sec(estimated); cost:%.3f(sec);\n", //perilipsi statistikon write
                        r ? "Random" : "Sequential", write_count,
                        write_time / write_count,
                        write_count / write_time,
                        write_time);

                printf(LINE1);


                 printf("|%s-Read    (done:%ld, found:%ld): %.6f sec/op; %.1f reads/sec(estimated); cost:%.3f(sec)\n",   //perilipsi statistikon read
                         r ? "Random" : "Sequential", read_count,
                         read_count, // Assuming all reads were successful
                         read_time / read_count,
                         read_count / read_time,
                         read_time);

                 printf(LINE);                                      //pernoume tin etoimi dilosi grammis stin bench.h
                 printf("Write-Percentage: %d%%\n", write_percent);
                 printf("Read-Percentage: %d%%\n", read_percent);
                 printf(LINE1);                                     //pernoume tin etoimi dilosi grammis stin bench.h
                 printf("Total time: %.3f seconds\n", total_time);
                 }
            }
        return 0;
        }
        if (argc < 3) {
                fprintf(stderr,"Usage: db-bench <write | read> <count>\n");
                exit(1);
        }

        if (strcmp(argv[1], "write") == 0) {
                int r = 0;

                count = atoi(argv[2]);
                _print_header(count);
                _print_environment();
                if (argc == 4)
                        r = 1;
                _write_test(count, r);
        } else if (strcmp(argv[1], "read") == 0) {
                int r = 0;

                count = atoi(argv[2]);
                _print_header(count);
                _print_environment();
                if (argc == 4)
                        r = 1;

                _read_test(count, r);
        } else {
                fprintf(stderr,"Usage: db-bench <write | read> <count> <random>\n");
                exit(1);
        }

        return 1;
}
```

1. Στο αρχείο bench.c προσθέσαμε μια νέα διακλάδωση που ενεργοποιείτε όταν ο χρήστης εισάγει την εντολή “write_read” με τουλάχιστον 5 παραμέτρους. Αρχικά δηλώνουμε την μεταβλητή r που χρησιμοποιείτε ως δείκτης (flag) για να ελέγξει αν η λειτουργία θα είναι σειριακή (sequential) ή τυχαία (random). Επόμενος δηλώνουμε μεταβλητές για τους χρόνους. Μετράμε τον αρχικό χρόνο και περνούμε τις τιμές από το τερματικό (αριθμό λειτουργιών, ποσοστό write, ποσοστό read). Στην συνέχεια υπάρχει έλεγχος αν εισάγει ο χρήστης στο τερματικό random αν ναι η μεταβλητή r (flag) ισούται με 1. Ακόμη εκτυπώνουμε τις δυο συναρτήσεις με τα στοιχεία και υπολογίζουμε τον νέο αριθμών λειτουργιών (write, read) βάση των ποσοστό που εισάγει ο χρήστης.

2. Δημιουργούμε μια διεργασία child (παιδί) για την εκτέλεση των write μέσω της _write_test() που καλείτε από την kiwi.c. Η διεργασία parent(γονέας) περιμένει να τελειώσει το write(child process) και μετά δημιουργεί ακόμη μια διεργασία child(παιδί) για το read, κάνουμε και έλεγχό αν το ποσοστό read που έχει εισαχθεί είναι μεγαλύτερο από το ποσοστό write και αν χρειαστεί περιορίζεται ο αριθμός στην τιμή του write. Εκτελείτε η συνάρτηση _read_test() από την kiwi.c και η διεργασία parent(γονέας) περιμένει την ολοκλήρωση της, επομένως υπολογίζονται οι τελικοί χρόνοι και εκτυπώνουμε λεπτομερείς στατιστικά του προγράμματος.

3. Υπάρχει σχόλιο σε κάθε γραμμή για ποιο αναλυτική επεξήγηση .

Κώδικας kiwi.c:

kiwi.c

```c
#include <string.h>
#include "../engine/db.h"
#include "../engine/variant.h"
#include "bench.h"

#define DATAS ("testdb")

void _write_test(long int count, int r)
{
        int i;
        double cost;
        long long start,end;
        Variant sk, sv;
        DB* db;
        char key[KSIZE + 1];
        char val[VSIZE + 1];
        char sbuf[1024];

        memset(key, 0, KSIZE + 1);
        memset(val, 0, VSIZE + 1);
        memset(sbuf, 0, 1024);

        db = db_open(DATAS);

        start = get_ustime_sec();
        for (i = 0; i < count; i++) {
                if (r)
                        _random_key(key, KSIZE);
                else
                        snprintf(key, KSIZE, "key-%d", i);
                fprintf(stderr, "%d adding %s\n", i, key);
                snprintf(val, VSIZE, "val-%d", i);

                sk.length = KSIZE;
                sk.mem = key;
                sv.length = VSIZE;
                sv.mem = val;

                db_add(db, &sk, &sv);
                if ((i % 10000) == 0) {
                        fprintf(stderr,"random write finished %d ops%30s\r",
                                        i,
                                        "");

                        fflush(stderr);
                }
        }

        db_close(db);

        end = get_ustime_sec();
        cost = end -start;

        printf(LINE);
        if(r==1){                                                                                    //added if r==1 random write else sequentia
                printf("|Random-Write   (done:%ld): %.6f sec/op; %.1f writes/sec(estimated); cost:%.3f(sec);\n"
                        ,count, (double)(cost / count)
                        ,(double)(count / cost)
                        ,cost);
        }else{                                                                                       //ektiposi Sequential anti Random
                printf("|Sequential-Write  (done:%ld): %.6f sec/op; %.1f writes/sec(estimated); cost:%.3f(sec);\n"
                        ,count, (double)(cost / count)
                        ,(double)(count / cost)
                        ,cost);
        }
}

void _read_test(long int count, int r)
{
        int i;
        int ret;
        int found = 0;
        double cost;
        long long start,end;
        Variant sk;
        Variant sv;
        DB* db;
        char key[KSIZE + 1];

        db = db_open(DATAS);
        start = get_ustime_sec();
        for (i = 0; i < count; i++) {
                memset(key, 0, KSIZE + 1);

                /* if you want to test random write, use the following */
                if (r)
                        _random_key(key, KSIZE);
                else
                        snprintf(key, KSIZE, "key-%d", i);
                fprintf(stderr, "%d searching %s\n", i, key);
                sk.length = KSIZE;
                sk.mem = key;
                ret = db_get(db, &sk, &sv);
                if (ret) {
                        //db_free_data(sv.mem);
                        found++;
```

```
        } else {
                INFO("not found key#%s",
                                sk.mem);
    }

        if ((i % 10000) == 0) {
                fprintf(stderr,"random read finished %d ops%30s\r",
                                i,
                                "");

                fflush(stderr);
        }
    }

    db_close(db);

    end = get_ustime_sec();
    cost = end - start;
    printf(LINE);
    if(r==1){                                                                                                    //added if r==1 random read else sequential
        printf("|Random-Read    (done:%ld, found:%d): %.6f sec/op; %.1f reads /sec(estimated); cost:%.3f(sec)\n",
                count, found,
                (double)(cost / count),
                (double)(count / cost),
                cost);
    }else{
        printf("|Sequential-Read  (done:%ld, found:%d): %.6f sec/op; %.1f reads /sec(estimated); cost:%.3f(sec)\n",     //ektiposi Sequential anti Random
                count, found,
                (double)(cost / count),
                (double)(count / cost),
                cost);
    }
}
```

1. Η μόνη αλλαγή που έχουμε κάνει στο αρχείο kiwi.c είναι για την σωστή εμφάνιση αποτελεσμάτων (sequential ή random), βάλαμε την εντολή printf μέσα σε μια δομή διακλαδώσεις που ελέγχουμε την τιμή r (flag) αν είναι 0 (false) ή 1 (true). Αν ισούται με 1 θα εκτυπώσει random αλλιώς sequential. (και στις δυο συναρτήσεις (_write_test & _read__test) είναι ακριβώς η ίδια αλλαγή)

Εξοδοι Εκτελεσης:

TERMINAL COMMAND:

1) ./kiwi-bench write_read 100000 50 50

```
------------------------------------------------------------------------------------------------------
|Sequential-Write       (done:50000): 0.000012 sec/op; 83966.6 writes/sec(estimated); cost:0.595(sec);
------------------------------------------------------------------------------------------------------
|Sequential-Read    (done:50000, found:50000): 0.000006 sec/op; 162082.4 reads/sec(estimated); cost:0.308(sec)
+-----------------------------+----------------+-----------------------------+-------------------+
Write-Percentage: 50%
Read-Percentage: 50%
------------------------------------------------------------------------------------------------------
Total time: 0.904 seconds
```

2) ./kiwi-bench write_read 100000 50 50 random

```
------------------------------------------------------------------------------------------------------
|Random-Write   (done:50000): 0.000011 sec/op; 91813.9 writes/sec(estimated); cost:0.545(sec);
------------------------------------------------------------------------------------------------------
|Random-Read    (done:50000, found:50000): 0.000016 sec/op; 61068.7 reads/sec(estimated); cost:0.819(sec)
+-----------------------------+----------------+-----------------------------+-------------------+
Write-Percentage: 50%
Read-Percentage: 50%
------------------------------------------------------------------------------------------------------
Total time: 1.364 seconds
```

3) ./kiwi-bench write_read 100000 70 30

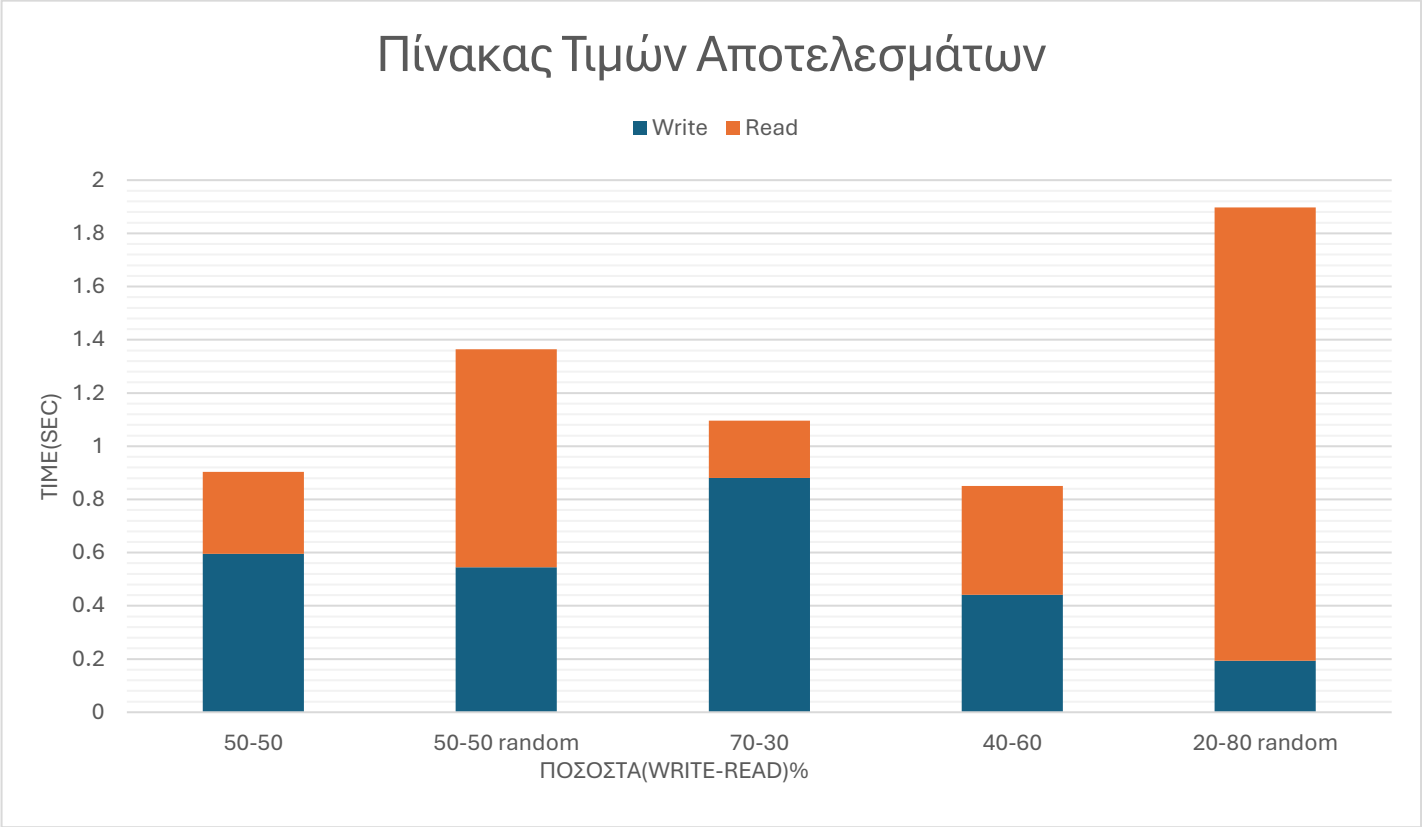
```
------------------------------------------------------------------------------------------------------
|Sequential-Write       (done:70000): 0.000013 sec/op; 79498.8 writes/sec(estimated); cost:0.881(sec);
------------------------------------------------------------------------------------------------------
|Sequential-Read    (done:30000, found:30000): 0.000007 sec/op; 139438.9 reads/sec(estimated); cost:0.215(sec)
+-----------------------------+----------------+-----------------------------+-------------------+
Write-Percentage: 70%
Read-Percentage: 30%
------------------------------------------------------------------------------------------------------
Total time: 1.096 seconds
```

4) ./kiwi-bench write_read 100000 40 60

```
-----------------------------------------------------------------------------------------------
|Sequential-Write       (done:40000): 0.000011 sec/op; 90422.7 writes/sec(estimated); cost:0.442(sec);
-----------------------------------------------------------------------------------------------
|Sequential-Read    (done:60000, found:60000): 0.000007 sec/op; 147007.7 reads/sec(estimated); cost:0.408(sec)
+-----------------------------+----------------+------------------------------+-------------------+
Write-Percentage: 40%
Read-Percentage: 60%
-----------------------------------------------------------------------------------------------
Total time: 0.851 seconds
```

5) ./kiwi-bench write_read 100000 20 80 random

```
-----------------------------------------------------------------------------------------------
|Random-Write   (done:20000): 0.000010 sec/op; 103327.7 writes/sec(estimated); cost:0.194(sec);
-----------------------------------------------------------------------------------------------
|Random-Read    (done:80000, found:80000): 0.000021 sec/op; 46961.2 reads/sec(estimated); cost:1.704(sec)
+-----------------------------+----------------+------------------------------+-------------------+
Write-Percentage: 20%
Read-Percentage: 80%
-----------------------------------------------------------------------------------------------
Total time: 1.897 seconds
```



Το random (50-50 random και 20-80 random) απαιτει περισσοτερο χρονο συνολικα σε συγκριση με την ακολουθιακη προσβαση με τα ιδια ποσοστα. Επισεις το 40-60 παρουσιάζει τον μικρότερο συνολικό χρόνο εκτέλεσης.

Εντολη Make:

```
myy601@myy601lab1:~/kiwi/kiwi-source$ make
cd engine && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/engine'
    CC db.o
    CC memtable.o
    CC indexer.o
    CC sst.o
    CC sst_builder.o
    CC sst_loader.o
    CC sst_block_builder.o
    CC hash.o
    CC bloom_builder.o
    CC merger.o
    CC compaction.o
    CC skiplist.o
    CC buffer.o
    CC arena.o
    CC utils.o
    CC crc32.o
    CC file.o
    CC heap.o
    CC vector.o
    CC log.o
    CC lru.o
    AR libindexer.a
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/engine'
cd bench && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/bench'
gcc -g -ggdb -Wall -Wno-implicit-function-declaration -Wno-unused-but-set-variable bench.c kiwi.c -L ../engine -lindexer -lpthread -lsnappy -o kiwi-bench
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/bench'
```